

NYCU DL Lab5 MaskGIT for Image Inpainting

313551073 顏琦恩

I. Introduction

In this lab, I implemented the MaskGit for the inpainting task from the paper Maskgit: Masked generative image transformer. For the first stage of the MastGit, I used the pretrained weight of VQGAN provided by TA. As for the second stage, I implemented the multi-head attention of the bidirectional transformer and the MaskGit model. I used the dataset provided by TA to train my model.

II. Implementation Details

A. The details of your model (Multi-Head Self-Attention)

For the multi-head self-attention, I first calculated the query, key, and value by feeding input into a linear neural network, and then transposed the q, k, and v to the right shape to do self-attention to each head. After going through the softmax and the dropout layer, I applied a linear layer to produce the final output.

```
6 class MultiHeadAttention(nn.Module):
7     def __init__(self, dim=768, num_heads=16, attn_drop=0.1):
8         super(MultiHeadAttention, self).__init__()
9         self.dim = dim
10        self.num_heads = num_heads
11        self.attn_drop = attn_drop
12
13        self.w_q = nn.Linear(dim, dim)
14        self.w_k = nn.Linear(dim, dim)
15        self.w_v = nn.Linear(dim, dim)
16
17        self.softmax = nn.Softmax(dim=-1)
18        self.dropout = nn.Dropout(attn_drop)
19
20        self.w_o = nn.Linear(dim, dim)
21
22    def forward(self, x):
23        # reference:
24        # https://github.com/CyberZHG/torch-multi-head-attention/blob/master/torch_multi_head_attention/multi_head_attention.py
25        # https://github.com/jadore801120/attention-is-all-you-need-pytorch/blob/master/transformer/SubLayers.py
26        batch_size = x.shape[0]
27
28        query = self.w_q(x)
29        key = self.w_k(x)
30        value = self.w_v(x)
31
32        query = query.view(batch_size, -1, self.num_heads, self.dim // self.num_heads).transpose(1, 2)
33        key = key.view(batch_size, -1, self.num_heads, self.dim // self.num_heads).transpose(1, 2)
34        value = value.view(batch_size, -1, self.num_heads, self.dim // self.num_heads).transpose(1, 2)
35
36        # Scaled Dot-Product Attention
37        matmul_qk = torch.matmul(query, key.transpose(-2, -1))
38        dk = key.size()[-1]
39        attention_weights = self.softmax(matmul_qk / (dk ** 0.5))
40        dropout_attention_weights = self.dropout(attention_weights)
41        output = torch.matmul(dropout_attention_weights, value)
42
43        output = output.transpose(1, 2).contiguous().view(batch_size, -1, self.dim)
44        output = self.w_o(output)
45        return output
```

Fig. 1 Details of Multi-Head Self-Attention

B. The details of your stage2 training (MVTM, forward, loss)

In the MVTM, I inputted the masked token into the bidirectional transformer and predicted the visual tokens. The following figure is the code of MVTM.

```
68     def forward(self, x):
69
70         _, z_indices = self.encode_to_z(x) #ground truth
71         mask = torch.bernoulli(0.5 * torch.ones(z_indices.shape, device=z_indices.device)).bool()
72
73         masked_indices = self.mask_token_id * torch.ones_like(z_indices, device=z_indices.device)
74         new_indices = mask * masked_indices + (~mask) * z_indices
75
76         logits = self.transformer(new_indices) #transformer predict the probability of tokens
77         return logits, z_indices
```

Fig. 2 Details of MVTM

For the training process, I used cross-entropy to calculate the loss between the visual tokens VQGAN generated and the predicted tokens generated by the bidirectional transformer.

```
19 class TrainTransformer:
20     def __init__(self, args, MaskGit_CONFIGS):
21         self.args = args
22         self.model = VQGANTransformer(MaskGit_CONFIGS["model_param"]).to(device=args.device)
23         self.optim, self.scheduler = self.configure_optimizers()
24         self.prepare_training()
25
26     @staticmethod
27     def prepare_training():
28         os.makedirs("transformer_checkpoints", exist_ok=True)
29
30     def train_one_epoch(self, data_loader):
31         self.model.train()
32         epoch_loss = []
33         for i, data in enumerate(tqdm(data_loader)):
34             logits, z_indices = self.model(data.to(self.args.device))
35             loss = F.cross_entropy(logits.reshape(-1, logits.size(-1)), z_indices.reshape(-1))
36             epoch_loss.append(loss.item())
37             loss.backward()
38             if i % args.accum_grad == 0:
39                 self.optim.step()
40                 self.optim.zero_grad()
41
42         torch.save(self.model.transformer.state_dict(), self.args.last_checkpoint_path)
43         return sum(epoch_loss) / len(epoch_loss)
```

Fig. 3 Details of training section

C. The details of your inference for inpainting task (iterative decoding)

The inpainting process is split into 3 parts, which is the predict section, the sample section, and the mask schedule section. I first predict the probabilities of the given mask token, and then use the probability to sample a possible token from the codebook. During the process, I applied the mask schedule to improve the model performance, which contains the linear schedule, the cosine schedule, and the square schedule.

```

82 @torch.no_grad()
83 def inpainting(self, z_indices, mask, mask_num, ratio):
84     device = mask.device
85     mask_indices = mask * self.mask_token_id + (~mask) * z_indices
86     logits = self.transformer(mask_indices)
87     #Apply softmax to convert logits into a probability distribution across the last dimension.
88     prob = torch.softmax(logits, dim=-1)
89
90     #FIND MAX probability for each token value
91     z_indices_predict_prob, z_indices_predict = prob.max(dim=-1)
92
93     ratio=ratio
94     #predicted probabilities add temperature annealing gumbel noise as confidence
95     g = torch.distributions.Gumbel(0, 1).sample(z_indices_predict_prob.shape).to(device) # gumbel noise
96     temperature = self.choice_temperature * (1 - ratio)
97     confidence = z_indices_predict_prob + temperature * g
98
99     #hint: If mask is False, the probability should be set to infinity, so that the tokens are not affected by the transformer's
100    #sort the confidence for the rank
101    #define how much the iteration remain predicted tokens by mask scheduling
102    #At the end of the decoding process, add back the original token values that were not masked to the predicted tokens
103    n = math.ceil(self.gamma(ratio) * mask_num)
104    confidence[~mask] = torch.inf
105    _, idx = confidence.topk(n, dim=-1, largest=False) #update indices to mask only smallest n token
106    mask_bc = torch.zeros(z_indices.shape, dtype=torch.bool, device= device)
107    mask_bc = mask_bc.scatter_(dim= 1, index= idx, value= True)
108    return z_indices_predict, mask_bc

```

Fig. 4 Details of iterative decoding

III. Discussion

The paper mentioned that the cosine mask schedule has the best FID under the same setting of the three mask schedules, but in my experiment, the linear mask schedule achieves the best FID under the setting: sweet spot set to 8, iterations set to 10. The following figures show the screenshots of FID results of three mask schedules.

Mask schedule	FID	
Cosine	100%  15/15	
	100%  15/15	
	FID: 40.10740695188582	
Linear	100%  15/15	
	100%  15/15	
	FID: 39.530614184376674	
Square	100%  15/15	
	100%  15/15	
	FID: 39.89495463394445	

IV. Experiment

A. Prove your code implementation is correct

- Mask in latent domain



- Predicted image



We can see that the mask is reduced gradually following the mask schedule, and the mask on the predicted image is also removed from the image.

B. The Best FID Score

- Screenshot

```
100%|████████████████████████████████████████████████████████████████████████████████| 15/15 [00:02<00:00, 6.99it/s]
100%|████████████████████████████████████████████████████████████████████████████████| 15/15 [00:01<00:00, 9.61it/s]
FID: 39.530614184376674
```

After testing, using the cosine mask schedule results in the lowest FID.

- Masked Images v.s MaskGIT Inpainting Results v.s Ground Truth



- Experiment setting

Training	
Parameter	Set Value
num_workers	4
batch_size	48
epochs	100
learning_rate	0.0001
Inferencing	
Parameter	Set Value
batch-size	1
num_workers	4
sweet-spot	8
total-iter	10